

Date:
Student Name:
Student ID:

AI Workshop #11

Deep Learning with Convolutional Neural Networks (CNNs)

11.1. Introduction

In this workshop, we are going to learn about **Deep learning** and **Convolutional Neural Networks (CNNs)**.

CNNs have gained a lot of momentum over the last few years, especially in the field of image recognition.

We will talk about the architecture of CNNs and the type of layers used inside.

We are going to see how to use a package called TensorFlow.

We will build a perceptron-based linear regressor.

We are going to learn how to build an image classifier using a single-layer neural network.

We will then build an image classifier using a CNN.

Image classifiers have many applications. It's a fancy name, but it's just the ability of computers to discern what an object is.

For example, you might build a classifier that determines if something is a hotdog or not a hotdog.

This is a lighthearted example, but image classifiers can also have life-or-death applications.

Picture a drone that has image classification software embedded in it and it can distinguish between civilians and enemy combatants. No mistakes can be made in that sort of case.

By the end of this workshop, you will have a better understanding of these topics:

- The basics of CNNs
- The architecture of CNNs
- The types of layers in a CNN
- Building a perceptron-based linear regressor
- Building an image classifier using a single-layer neural network
- Building an image classifier using a CNN

Let's begin by running through the basics.

11.2 The basics of Convolutional Neural Networks

CNNs in general, and **Generative Adversarial Networks (GANs)** in particular, have been in the news lately.

A GAN is a class of CNN developed by Ian Goodfellow and his colleagues initially in 2014.

In GANs, two neural networks compete against each other in a game (in a game theory sense).

Given a dataset, a GAN learns to create new data examples similar to the training set.

For example, it might be a little slow but there is a website that will generate faces of people that don't exist.

We'll let your imagination run wild but it would certainly be possible to create a film using some of these generated "humans" to star in the movie.

There is other research to try to solve the converse.

Given an image, can we determine if it's a GAN-generated image or a real person? You can play around with the website here:

<https://thispersondoesnotexist.com/> (<https://thispersondoesnotexist.com/>).

To use it, just keep on refreshing the page and it will generate a new image every time.

GANs were originally created as a generative model for unsupervised learning.

GANs have also proven useful for semi-supervised learning, supervised learning, and reinforcement learning.

Yann LeCun, one of the giants of AI, calls GANs, the most interesting idea in the last 10 years in ML. Let's consider some other use cases and applications of GANs.

Generating more example data using GANs – Data is the ingredient in ML.

In some cases, it is not possible to get enough data to feed to a model.

Using GANs to generate more input data is a good way to generate additional quality data to feed into a model.

Security – ML has provided a boost to many industries.

Regardless of the market sector, cyber security is always "top of mind" for the C-suite.

GANs are used by some security vendors to handle cyberattacks.

In simple terms, the GAN creates fake intrusions, and these intrusions are used to train models to identify these threats, enabling us to thwart genuine versions of these attacks.

Data manipulation – GANs can be used for "pseudo style transfers," that is, modifying some dimensions of the example without completely modifying it.

GANs can be used in voice applications. Given a speech, a GAN can be trained to reproduce famous voices.

In some famous examples, videos of Trump, Obama, or the Mona Lisa were modified using GANs and the developers made these figures say phrases they never said.

They can be quite realistic. Or video or images can be changed to look like different people.

Here is an example of the interposition of Nicolas Cage's face on top of an image of President Donald Trump created by MIT:



These techniques can be ported to other domains such as natural language processing, speech processing, and so on.

As an example, the GAN might tweak a sentence slightly, changing the meaning of the sentence.

Privacy – As part of their security strategy, many companies want to keep some of the data private and confidential.

Obvious examples would come in the form of defense and military applications.

GANs can be used while encrypting data; for example, to generate one-time throwaway keys.

In 2016, Google began research in order to better utilize GANs.

The basic idea was to have one network creating a passkey and another network trying to crack it.

We saw how neural networks work throughout the previous two chapters.

Neural networks consist of neurons that have weights and biases.

These weights and biases are tuned during the training process to come up with a good learning model.

Each neuron receives a set of inputs, processes it in some way, and then outputs a value.

If we build a neural network with many layers, it's called a deep neural network.

The branch of AI dealing with these deep neural networks is referred to as deep learning.

One of the main disadvantages of ordinary neural networks is that they ignore the structure of input data.

All data is converted to a single-dimensional array before feeding it into the network.

This might work well with numeric data, but things get difficult when we deal with images.

Let's consider grayscale images. These images are 2D structures and we know that the spatial arrangement of pixels has a lot of hidden information.

If we ignore this information, we will be losing a lot of underlying patterns.

This is where CNNs come into the picture.

CNNs take the 2D structure of the images into account when they process them.

CNNs are also made up of neurons consisting of weights and biases.

These neurons accept input data, process it, and then output something.

The goal of the network is to go from the raw image data in the input layer to the correct class in the output layer.

The difference between ordinary neural networks and CNNs is in the type of layers we use and how we treat the input data.

CNNs assume that the inputs are images, which allows them to extract properties specific to images.

This makes CNNs way more efficient in dealing with images.

11.3 Architecture of CNNs

When we are working with ordinary neural networks, we need to convert the input data into a single vector.

This vector acts as the input to the neural network, which then passes through the layers of the neural network.

In these layers, each neuron is connected to all the neurons in the previous layer.

It is also worth noting that the neurons within each layer are not connected to each other.

They are only connected to the neurons in the adjacent layers.

The last layer in the network is the output layer and it represents the final output.

If we use this structure for images, it will quickly become unmanageable.

For example, let's consider an image dataset consisting of 256×256 RGB images.

Since these are 3-channel images, there would be $256 * 256 * 3 = 196,608$ weights.

Note that this is just for a single neuron! Each layer will have multiple neurons, so the number of weights tends to increase rapidly.

This means that the model will now have an enormous number of parameters to tune during the training process.

Therefore, it quickly becomes quite complex and time-consuming.

Connecting each neuron to every neuron in the previous layer, called full connectivity, is clearly not going to work.

CNNs explicitly consider the structure of images when processing the data.

The neurons in CNNs are arranged in 3 dimensions – width, height, and depth. 4

Each neuron in the current layer is connected to a small patch of the output from the previous layer.

It's like overlaying an $N \times N$ filter on the input image.

This contrasts with a fully connected layer where each neuron is connected to all the neurons of the previous layer.

Since a single filter cannot capture all the nuances of the image, we do this M number of times to make sure we capture all the details. These M filters act as feature extractors.

If you look at the outputs of these filters, we can see that they extract features like edges, corners, and so on. This is true for the initial layers in the CNN.

As we progress through layers of the network, we will see that the latter layers extract higher-level features.

A CNN is a deep learning network. It is often used to recognize images.

Understanding how it recognizes an image will help in gaining an understanding about how they work.

Like any other neural network, it assigns weights and biases to elements in the image and is able to differentiate these elements from one another.

The pre-processing used in a CNN is less compared to other classification models.

When using primitive methods filters with enough training, CNNs can be trained to distinguish these filters and characteristics.

A CNN architecture, in its basic form, can be compared to the neurons and dendrites in the human brain, and it borrows its inspiration from the visual cortex.

Individual neurons respond to stimuli in restricted regions of the visual field.

This region is known as the Receptive Field. A grouping of these fields overlap with one another and thus cover the entire field of vision.

11.4 CNNs vs. perceptron neural networks

An image is a pixel-value matrix.

Why couldn't we just flatten the input image?

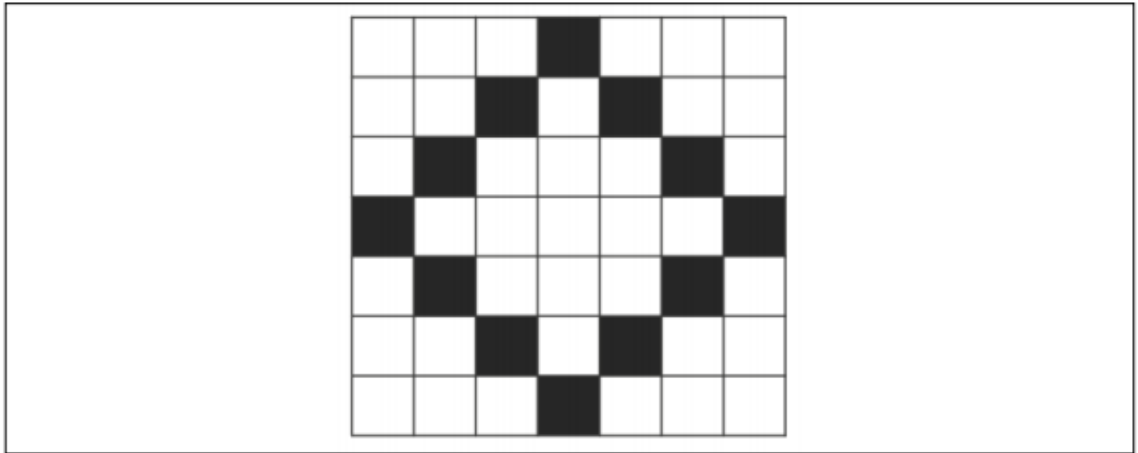
For example, a 7×7 image can be flattened into a 49×1 vector.

We can then use this flattened image as input to a perceptron-based neural network.

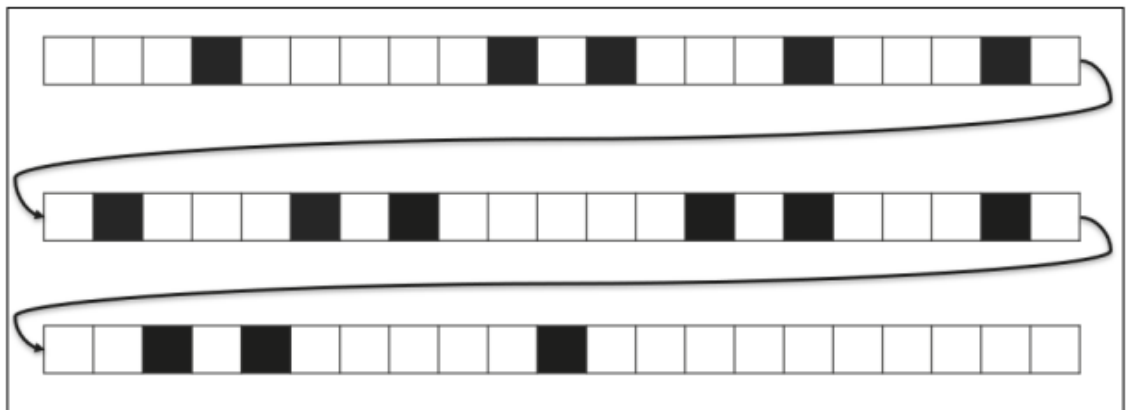
When using basic binary (black and white) inputs, this method might show an average precision score while performing prediction of classes, but would have little to no accuracy when it comes to complex images having pixel dependencies throughout.

Let's analyze this for a second to get some understanding by thinking about how humans process an image.

Think of an image that contains a diamond shape: ♦. Our brain can instantaneously process the image and realize that it's a diamond shape: ♦.



What happens if we flatten it?



It's not so easy to recognize, is it?

Despite this, it is the same information. Something similar happens when using traditional neural networks rather than CNNs.

The information that we had when the pixels were contiguous is now lost.

A CNN can capture spatial and temporal dependencies in an image through the application of relevant filters.

The CNN architecture performs better upon a dataset because of the reduction in the number of parameters and the fact that weights are reused.

Now that we have a better understanding of CNN architecture and how images are processed, let's think about the layers that comprise a CNN.

11.5 Types of layers in a CNN

CNNs typically use the following types of layers:

Input layer – This layer takes the raw image data as it is.

Convolutional layer – This layer computes the convolutions between the neurons and the various patches in the input.

If you need a quick refresher on image convolutions, you can check out this link:

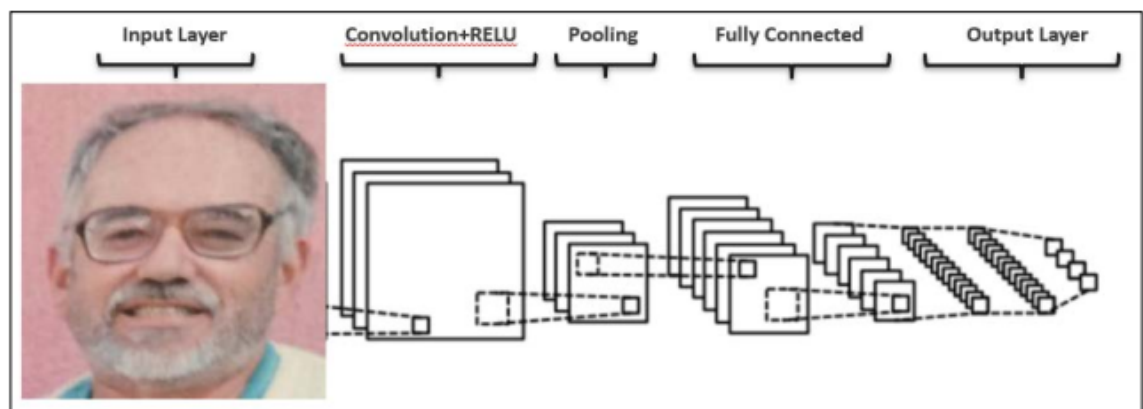
http://web.pdx.edu/~jduh/courses/Archive/geog481w07/Students/Ludwig_ImageConvolution.ppt
([http://web.pdx.edu/~jduh/courses/Archive/geog481w07/Students/Ludwig_ImageConvolution.p](http://web.pdx.edu/~jduh/courses/Archive/geog481w07/Students/Ludwig_ImageConvolution.ppt)

The convolutional layer basically computes the dot product between the weights and a small patch in the output of the previous layer.

Rectified Linear Unit layer – This layer applies an activation function to the output of the previous layer. This function is usually something like $\max(0, x)$. This layer is needed to add non-linearity to the network so that it can generalize well to any type of function.

Pooling layer – This layer samples the output of the previous layer resulting in a structure with smaller dimensions. Pooling helps us to keep only the prominent parts as we progress in the network. Max pooling is frequently used in the pooling layer, where we pick the maximum value in a given $K \times K$ window.

Fully Connected layer – This layer computes the output scores in the last layer. The resulting output is of the size $1 \times 1 \times L$, where L is the number of classes in the training dataset.



As we go from the input layer to the output layer in the network, the input image gets transformed from pixel values to the final class scores.

Many different architectures for CNNs have been proposed and it's an active area of research.

The accuracy and robustness of a model depends on many factors – the type of layers, depth of the network, the arrangement of various types of layers within the network, the functions chosen for each layer, training data, and so on.

11.6 Building a perceptron-based linear regressor

Before we build a CNN, let's set the stage with a more basic model and then see how we can improve using CNNs.

In this section we will see how to build a linear regression model using perceptrons. We have already seen linear regression in previous chapters, but this section is about building a linear regression model using a neural network approach.

We will be using TensorFlow in this workshop. It is a popular deep learning package that's widely used to build various real-world systems.

In this section, we will get familiar with how it works. Make sure to install it before you proceed. The installation instructions can be found [here](#):

1. Once you verify that it's installed, create a new Python file and import the following packages:

```
# The following code is using Tensorflow 2.17.0, if you are using
Tensorflow 1.X please refer to the AIWorkshop11_old.ipynb file
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

2. We will be generating some data points and see how to train a model using these data points. Define the number of data points to be generated:

```
# Define the number of points to generate
num_points = 1200
```

3. Define the parameters that will be used to generate the data. We will be using the model of a line: $y = mx + c$:

```
# Generate the data based on equation  $y = mx + c$ 
data = []
m = 0.2
c = 0.5
for i in range(num_points):
    # Generate 'x'
    x = np.random.normal(0.0, 0.8)
```

4. Generate some noise to add some variation in the data:

```
# Generate some noise
noise = np.random.normal(0.0, 0.04)
```

5. Compute the value of y using the equation:

```
# Compute 'y'
y = m*x + c + noise

data.append([x, y])
```

6. Once you finish iterating, separate the data into input and output variables:

```
# Separate x and y
x_data = [d[0] for d in data]
y_data = [d[1] for d in data]

# Convert to numpy arrays
x_data = np.array(x_data)
y_data = np.array(y_data)
```

7. Plot the data:

```
# Plot the generated data
plt.plot(x_data, y_data, 'ro')
plt.title('Input data')
plt.show()
```

8. Define Kera Model

```
# Define keras model
model = tf.keras.models.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(1)
])
```

9. Compile the model

```
# Compile the model
model.compile(optimizer='sgd', loss='mean_squared_error')
```

9. Start Iteration

```
# Start iterating
num_iterations = 10
for step in range(num_iterations):
    # Train the model
    model.fit(x_data.reshape(-1, 1), y_data, verbose=0)
```

11. Print the progress

```
# Print the progress
print('\nITERATION', step+1)
print('W =', model.layers[0].weights[0].numpy()[0][0])
print('b =', model.layers[0].weights[1].numpy()[0])
print('loss =', model.loss)
```

12. Plot the input data

```
# Plot the input data
plt.plot(x_data, y_data, 'ro')
```

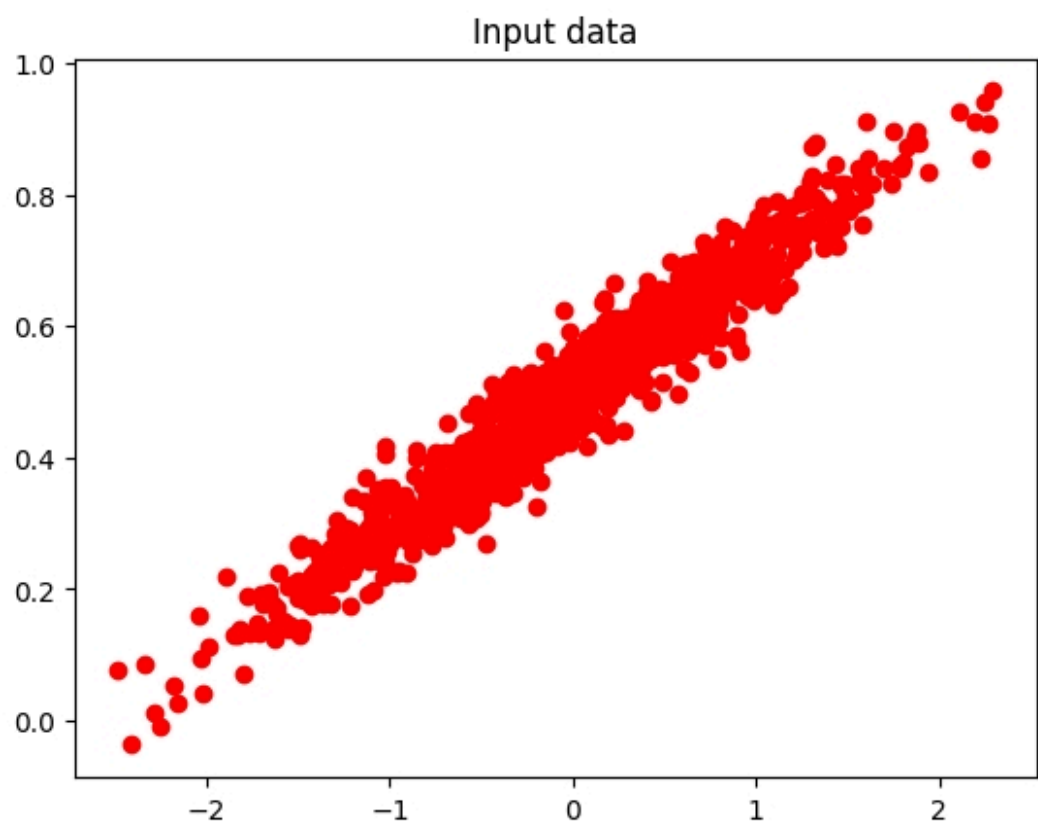
13. Plot the predicted output

```
# Plot the predicted output line  
plt.plot(x_data, model.predict(x_data.reshape(-1, 1)).flatten())
```

14. Set plotting parameters and the plots

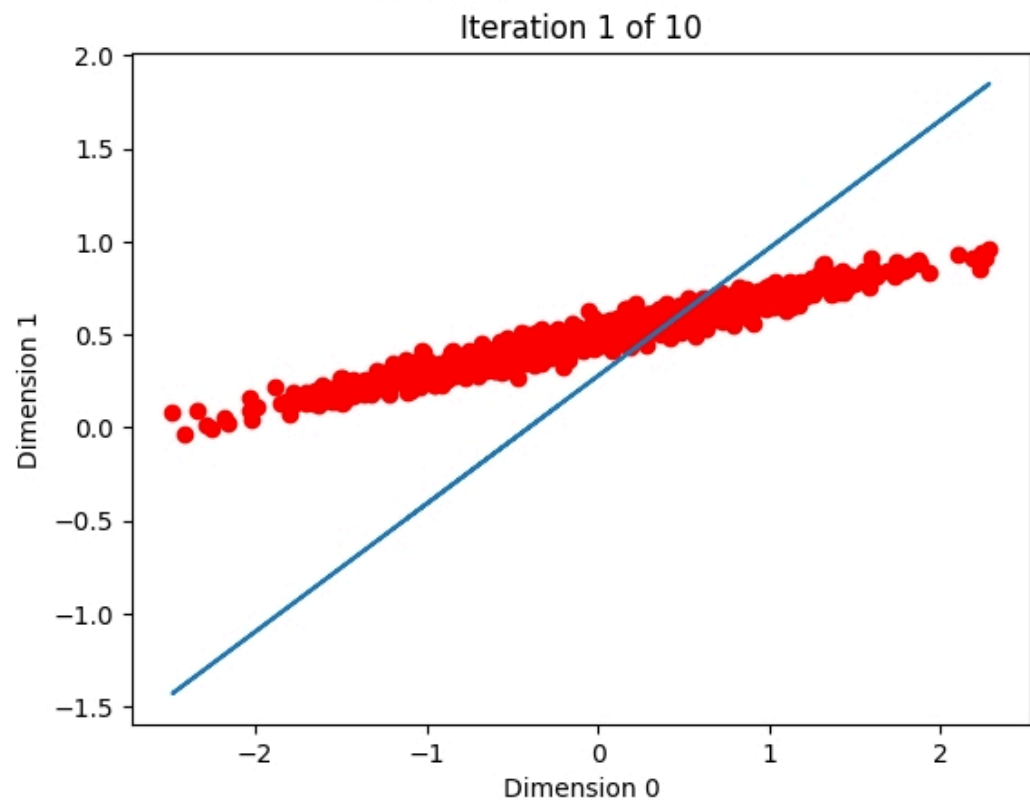
```
# Set plotting parameters  
plt.xlabel('Dimension 0')  
plt.ylabel('Dimension 1')  
plt.title('Iteration ' + str(step+1) + ' of ' + str(num_iterations))  
plt.show()
```

15. If you run the code, you should see following screenshot:



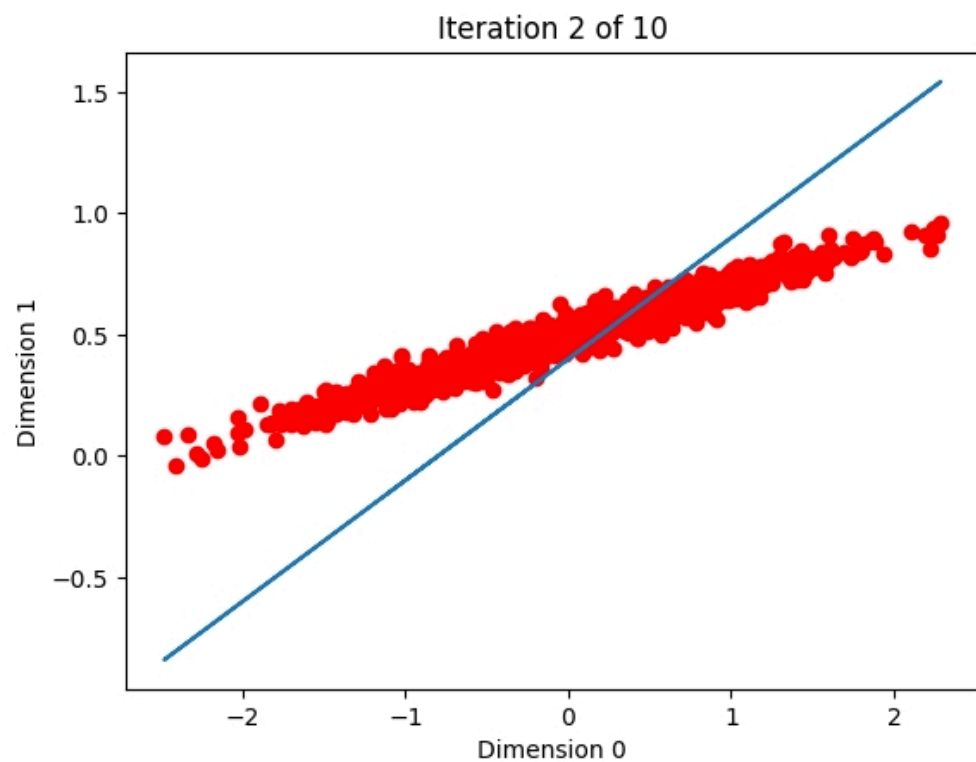
If you close this window, you will see the training process. The first iteration looks like this:

```
ITERATION 1
W = 0.68661916
b = 0.27619016
loss = mean_squared_error
38/38 ————— 0s 1ms/step
```



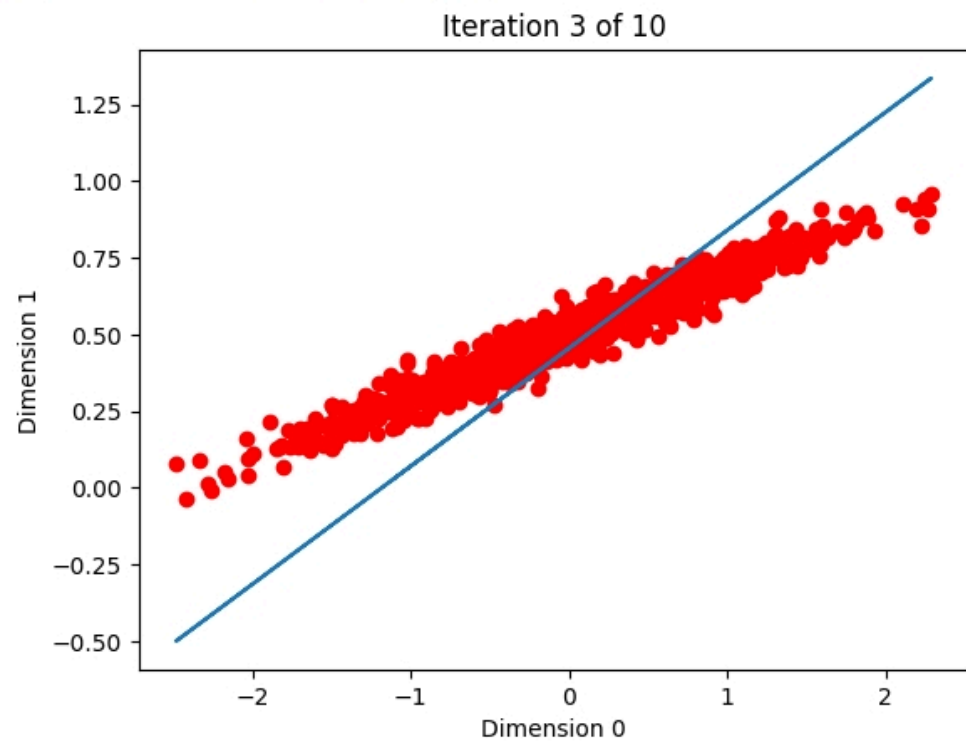
As we can see, the line is completely off. Close this window to go to the next iteration:

```
ITERATION 2
W = 0.4991615
b = 0.39932635
loss = mean_squared_error
38/38 ————— 0s 532us/step
```



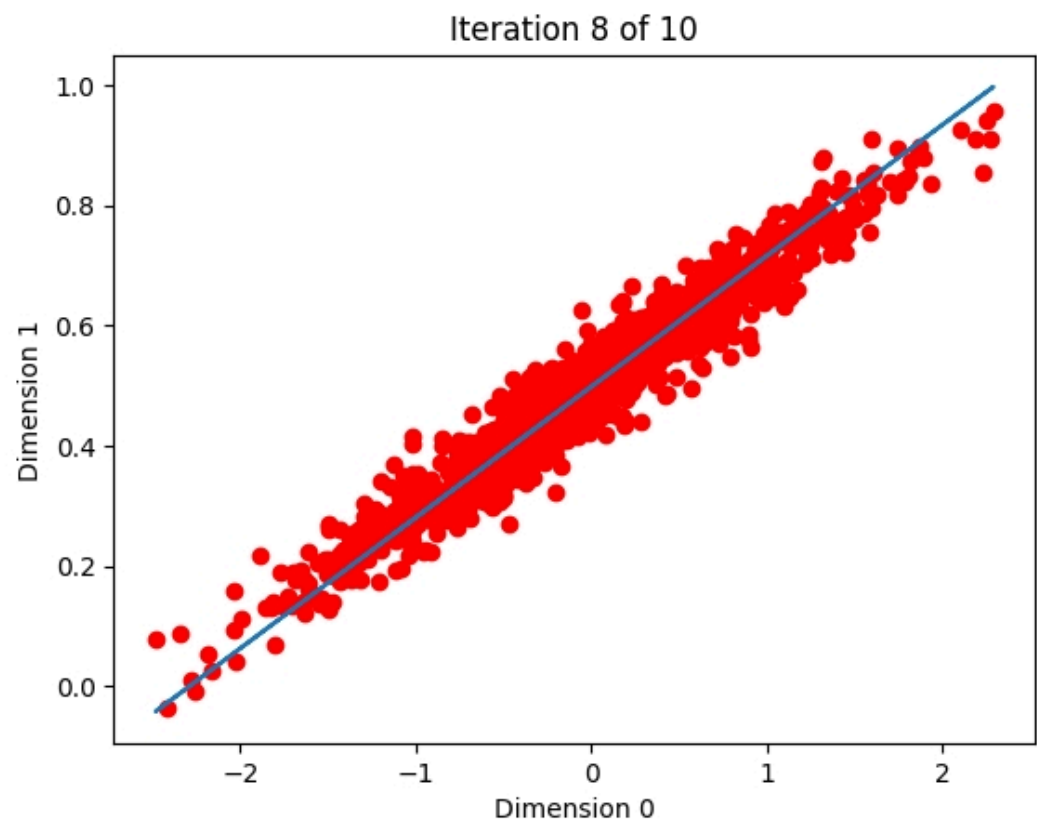
The line seems better, but it's still off. Let's close this window and continue iterating:

```
ITERATION 3  
W = 0.38455355  
b = 0.45505235  
loss = mean_squared_error  
38/38 ————— 0s 896us/step
```



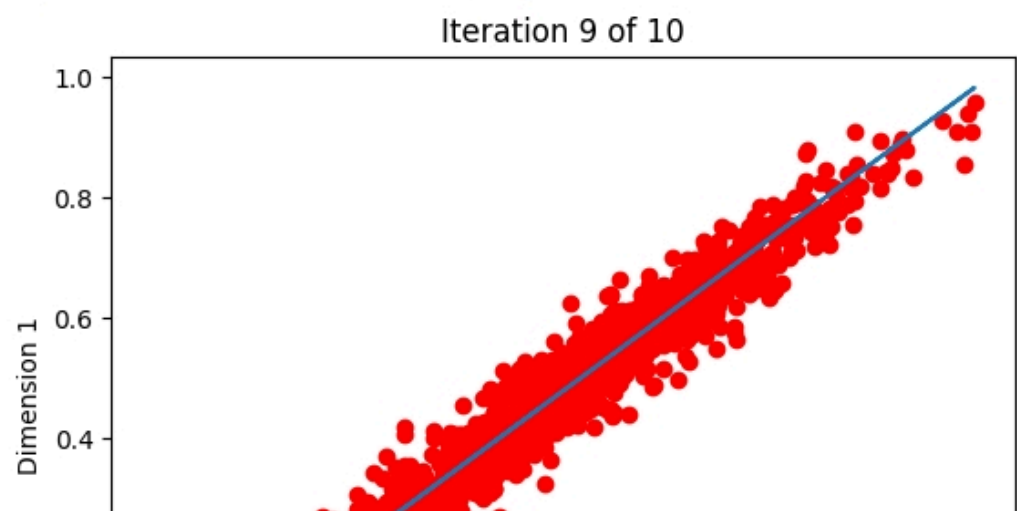
It looks like the line is getting closer to the real model. If you continue iterating like this, the model gets better. The 8th iteration looks like this:

```
ITERATION 8  
W = 0.2176193  
b = 0.49874514  
loss = mean_squared_error  
38/38 ————— 0s 603us/step
```



The 9th iteration looks like this:

```
ITERATION 9  
W = 0.21131709  
b = 0.4992491  
loss = mean_squared_error  
38/38 ————— 0s 437us/step
```





Workshop 11.1 : Linear Regression with CNN

1. Combine all the above code and named it as **LR_CNN.py**.
2. If everything is okay, you should see the above figures of iteration performance and performance results.
3. Save and submit the source code and these figures.
4. What is your findings from the training results?
5. Try this by using a NON-linear curve such as the famous **Sigmoid Curve** ($S(x)=1/(1+e^{-x})$) to see whether it also works. Save it as **NLR_CNN.py**,

17.7 Building an image classifier using a single-layer neural network

Let's see how to create a single-layer neural network using TensorFlow and use it to build an image classifier.

We will be using the MNIST image dataset <http://yann.lecun.com/exdb/mnist/> (<http://yann.lecun.com/exdb/mnist/>) build our system. It is a dataset containing images of handwritten digits.

Our goal is to build a classifier that can correctly identify the digit in each image.

2. Create a new Python file and import the following packages:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

import tensorflow as tf
from tensorflow.keras.datasets import mnist
```

3. Extract the MNIST image data.

```
# Get the MNIST data
batch_size = 90
(x_train, y_train), (x_test, y_test) = mnist.load_data()
train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))
test_dataset = tf.data.Dataset.from_tensor_slices((x_test, y_test))
train_db = train_dataset.shuffle(10000).batch(batch_size).repeat()
```

```
test_db = test_dataset.batch(batch_size)
```

4. Define the keras model.

```
# Define model
class Model(tf.keras.Model):
    def __init__(self):
        super(Model, self).__init__()
        self.flatten = tf.keras.layers.Flatten()
        self.dense = tf.keras.layers.Dense(10)

    def call(self, x):
        x = self.flatten(x)
        y = self.dense(x)
        return y

model = Model()
```

5. Define the gradient descent optimizer

```
# Define the gradient descent optimizer
optimizer = tf.keras.optimizers.SGD(0.5)
```

6. Define the loss function

```
# Define the loss function
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

7. Define the accuracy metric

```
# Define the accuracy metric
accuracy_fn = tf.keras.metrics.SparseCategoricalAccuracy()
```

8. Start training

```
# Start training
num_iterations = 1200
for i, (x_data, y_data) in enumerate(train_db.take(num_iterations)):
    with tf.GradientTape() as tape:
        # Train the model
        y = model(x_data)
        # Define the entropy loss
        loss = loss_fn(y_data, y)
    # Calculate the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    # Optimizing model parameters
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    # Update accuracy
    accuracy_fn.update_state(y_data, y)
```

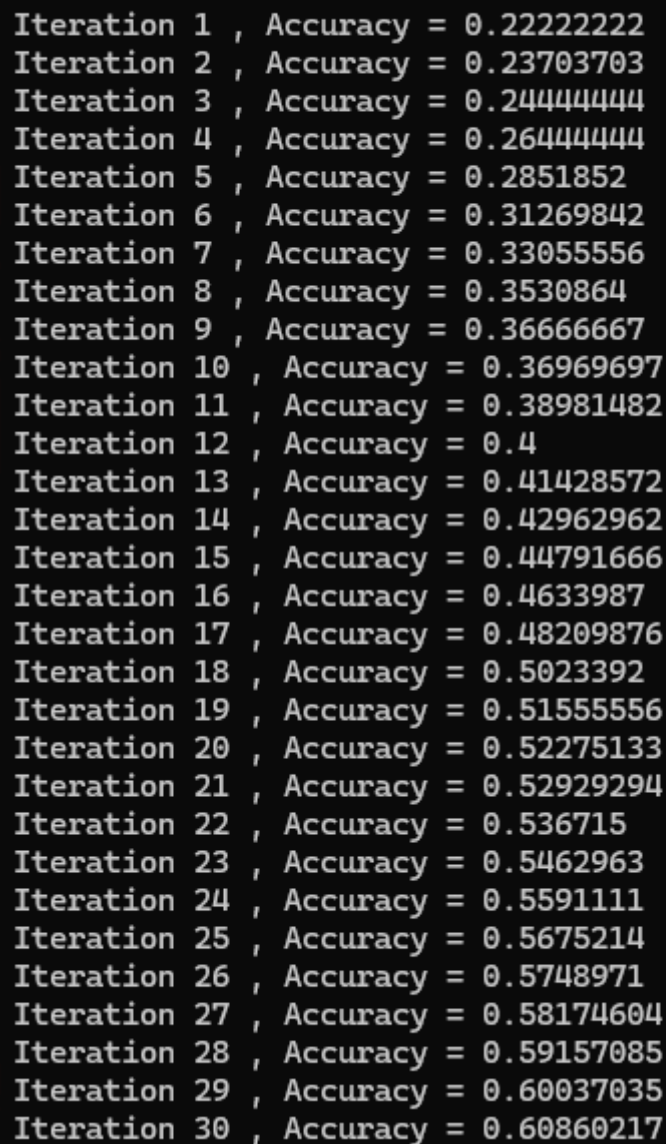
```
print('Iteration', i, ', Accuracy =', accuracy_fn.result().numpy())
```

9. Compute the accuracy using test data

```
# Compute the accuracy using test data
accuracy_fn.reset_state()
for x_data, y_data in test_db:
    y = model(x_data)
    accuracy_fn.update_state(y_data, y)
print('\nAccuracy =', accuracy_fn.result().numpy())
```

If you run the program, you will see training epochs. The speed of training is highly depended on you CPU/GPU capacity. Be patient.

The first 30 epochs are:



```
Iteration 1 , Accuracy = 0.22222222
Iteration 2 , Accuracy = 0.23703703
Iteration 3 , Accuracy = 0.24444444
Iteration 4 , Accuracy = 0.26444444
Iteration 5 , Accuracy = 0.2851852
Iteration 6 , Accuracy = 0.31269842
Iteration 7 , Accuracy = 0.33055556
Iteration 8 , Accuracy = 0.3530864
Iteration 9 , Accuracy = 0.36666667
Iteration 10 , Accuracy = 0.36969697
Iteration 11 , Accuracy = 0.38981482
Iteration 12 , Accuracy = 0.4
Iteration 13 , Accuracy = 0.41428572
Iteration 14 , Accuracy = 0.42962962
Iteration 15 , Accuracy = 0.44791666
Iteration 16 , Accuracy = 0.4633987
Iteration 17 , Accuracy = 0.48209876
Iteration 18 , Accuracy = 0.5023392
Iteration 19 , Accuracy = 0.51555556
Iteration 20 , Accuracy = 0.52275133
Iteration 21 , Accuracy = 0.52929294
Iteration 22 , Accuracy = 0.536715
Iteration 23 , Accuracy = 0.5462963
Iteration 24 , Accuracy = 0.5591111
Iteration 25 , Accuracy = 0.5675214
Iteration 26 , Accuracy = 0.5748971
Iteration 27 , Accuracy = 0.58174604
Iteration 28 , Accuracy = 0.59157085
Iteration 29 , Accuracy = 0.60037035
Iteration 30 , Accuracy = 0.60860217
```

The last section of epochs with the test accuracy:

```
C:\Windows\system32\cmd.e: X + v
Iteration 1174 , Accuracy = 0.8473988
Iteration 1175 , Accuracy = 0.84741515
Iteration 1176 , Accuracy = 0.84744096
Iteration 1177 , Accuracy = 0.847495
Iteration 1178 , Accuracy = 0.8475773
Iteration 1179 , Accuracy = 0.84757465
Iteration 1180 , Accuracy = 0.8475626
Iteration 1181 , Accuracy = 0.84756935
Iteration 1182 , Accuracy = 0.84753853
Iteration 1183 , Accuracy = 0.8475265
Iteration 1184 , Accuracy = 0.84757084
Iteration 1185 , Accuracy = 0.8476338
Iteration 1186 , Accuracy = 0.8476966
Iteration 1187 , Accuracy = 0.84774065
Iteration 1188 , Accuracy = 0.847794
Iteration 1189 , Accuracy = 0.84783787
Iteration 1190 , Accuracy = 0.847863
Iteration 1191 , Accuracy = 0.8478974
Iteration 1192 , Accuracy = 0.8479225
Iteration 1193 , Accuracy = 0.8479475
Iteration 1194 , Accuracy = 0.84800035
Iteration 1195 , Accuracy = 0.8479974
Iteration 1196 , Accuracy = 0.84799445
Iteration 1197 , Accuracy = 0.8480193
Iteration 1198 , Accuracy = 0.84803486
Iteration 1199 , Accuracy = 0.8480411
```



Workshop 11.2 : Image Classifier with Single-layer Neural Network

1. Combine all the above code and named it as **slnn.py**.
2. If everything is okay, you should see the above figures of iteration performance and performance results.
3. Save and submit the source code and these figures.
4. What is your findings from the training results?
5. How can you further improve the accuracy? Any suggestions?
6. Try to futher improve the acccuracy in terms of network parameters, network architecture and training epochs.
7. Plot your performance results and submit the source codes and figures.

11.8 Building an image classifier using a Convolutional Neural Network

The image classifier in the previous section didn't perform that well.

Getting 92.1% on the MNIST dataset is relatively easy. Let's see how we can use CNNs to achieve a much higher accuracy.

We will build an image classifier using the same dataset, but with a CNN instead of a single-layer neural network.

1. Create a new Python file and import the following packages:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
import argparse

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras import Input
```

2. Get the MNIST data

```
# Get the MNIST data
batch_size = 75
(x_train, y_train), (x_test, y_test) = mnist.load_data()
train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))
test_dataset = tf.data.Dataset.from_tensor_slices((x_test, y_test))
train_db = train_dataset.shuffle(10000).batch(batch_size).repeat()
test_db = test_dataset.batch(batch_size)
```

3. Define CNN model with convolution, pooling layers

```
# Define model
class Model(tf.keras.Model):
    def __init__(self):
        super(Model, self).__init__()
        self.model = tf.keras.Sequential([
            Input(shape=(28, 28, 1)),
            tf.keras.layers.Conv2D(64, (5, 5), padding='same',
activation='relu'),
            tf.keras.layers.MaxPool2D((2, 2), strides=2, padding='same'),
            tf.keras.layers.Conv2D(64, (5, 5), padding='same',
activation='relu'),
            tf.keras.layers.MaxPool2D((2, 2), strides=2, padding='same'),
            tf.keras.layers.Flatten(),
            tf.keras.layers.Dense(1024, activation='relu'),
            tf.keras.layers.Dropout(0.5),
            tf.keras.layers.Dense(10)
        ])

    def call(self, inputs):
        return self.model(inputs)
```

```
model = Model()
```

4. Define the optimizer

```
# Define the optimizer  
optimizer = tf.keras.optimizers.Adam(1e-4)
```

5. Define the loss function

```
# Define the loss function  
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

6. Define the accuracy metric

```
# Define the accuracy metric  
accuracy_fn = tf.keras.metrics.SparseCategoricalAccuracy()
```

7. Start training

```
# Start training  
num_iterations = 100  
print('\nTraining the model....')  
for i in range(num_iterations):  
    accuracy_fn.reset_state()  
    # Get the next batch of images  
    for j, (x_data, y_data) in enumerate(train_db.take(1)):  
        with tf.GradientTape() as tape:  
            # Train the model  
            y = model(x_data, training=True)  
            # Define the entropy loss  
            loss = loss_fn(y_data, y)  
        # Calculate the gradient  
        grads = tape.gradient(loss, model.trainable_variables)  
        # Optimizing model parameters  
        optimizer.apply_gradients(zip(grads, model.trainable_variables))  
        # Update accuracy  
        accuracy_fn.update_state(y_data, y)  
    print('Iteration', i, ', Accuracy =', accuracy_fn.result().numpy())
```

8. Compute accuracy using test data

```
# Compute accuracy using test data  
accuracy_fn.reset_state()  
for x_data, y_data in test_db:  
    y = model(x_data)  
    accuracy_fn.update_state(y_data, y)  
print('Test accuracy =', accuracy_fn.result().numpy())
```

If you run the program, you will see the following outputs. The speed of training is highly depended on you CPU/GPU capacity. Be patient.

Here is the first 30 training epochs:

```
C:\Windows\system32\cmd.e: X + v
Training the model....
Iteration 0 , Accuracy = 0.093333334
Iteration 1 , Accuracy = 0.14666666
Iteration 2 , Accuracy = 0.25333333
Iteration 3 , Accuracy = 0.25333333
Iteration 4 , Accuracy = 0.22666667
Iteration 5 , Accuracy = 0.37333333
Iteration 6 , Accuracy = 0.38666666
Iteration 7 , Accuracy = 0.53333336
Iteration 8 , Accuracy = 0.38666666
Iteration 9 , Accuracy = 0.5466667
Iteration 10 , Accuracy = 0.5466667
Iteration 11 , Accuracy = 0.6666667
Iteration 12 , Accuracy = 0.74666667
Iteration 13 , Accuracy = 0.73333335
Iteration 14 , Accuracy = 0.81333333
Iteration 15 , Accuracy = 0.68
Iteration 16 , Accuracy = 0.72
Iteration 17 , Accuracy = 0.70666665
Iteration 18 , Accuracy = 0.81333333
Iteration 19 , Accuracy = 0.74666667
Iteration 20 , Accuracy = 0.76
Iteration 21 , Accuracy = 0.74666667
Iteration 22 , Accuracy = 0.8
Iteration 23 , Accuracy = 0.88
Iteration 24 , Accuracy = 0.85333335
Iteration 25 , Accuracy = 0.76
Iteration 26 , Accuracy = 0.82666665
Iteration 27 , Accuracy = 0.84
Iteration 28 , Accuracy = 0.8
Iteration 29 , Accuracy = 0.82666665
Iteration 30 , Accuracy = 0.8933333
```

Here is the last 30 training epochs, together with the testing accuracy:

```
C:\Windows\system32\cmd.e: X + v
Iteration 70 , Accuracy = 0.88
Iteration 71 , Accuracy = 0.92
Iteration 72 , Accuracy = 0.88
Iteration 73 , Accuracy = 0.92
Iteration 74 , Accuracy = 0.9066667
Iteration 75 , Accuracy = 0.94666666
Iteration 76 , Accuracy = 0.9866667
Iteration 77 , Accuracy = 0.88
Iteration 78 , Accuracy = 0.9066667
Iteration 79 , Accuracy = 0.88
Iteration 80 , Accuracy = 0.9066667
Iteration 81 , Accuracy = 0.88
Iteration 82 , Accuracy = 0.9866667
Iteration 83 , Accuracy = 0.94666666
Iteration 84 , Accuracy = 0.92
Iteration 85 , Accuracy = 0.8666667
```



Workshop 11.3 : Image Classifier with CNN

1. Combine all the above code and named it as **cnn.py**.
2. If everything is okay, you should see the above figures of iteration performance and performance results.
3. Save and submit the source code and these figures.
4. What is your findings from the training results, as compared with the SLNN model?
5. Plot the training performance of CNN vs SLNN models.



11.9 Mini-project : Testing CNN with Yale Facial Database

One of the most important applications of CNN on pattern classification/identification is Human Faces.

Yale Facial database is one of the most famous facial database for the image classification and face recognition.



The database contains 165 GIF images of 15 subjects (subject01, subject02, etc.). There are 11 images per subject, one for each of the following facial expressions or configurations: center-light, w/glasses, happy, left-light, w/no glasses, normal, right-light, sad, sleepy, surprised, and wink. Note that the image "subject04.sad" has been corrupted and has been substituted by "subject04.normal".

One can access the Yale Facial dataset from here:

<https://www.kaggle.com/datasets/olgabelitskaya/yale-face-database>
(<https://www.kaggle.com/datasets/olgabelitskaya/yale-face-database>)

Or you can use my downloaded dataset in the workshop directory as well.

1. Study the Yale Face dataset and the data structure for data extraction.
2. First, using Single-Layered Neural Network for system training and testing, named it as **face_sltnn.py**.
3. Second, using Multi-Layered Neural Network for system training and testing, named it as **face_mltnn.py**.
4. Third, using CNN for system training and testing, named it as **face_cnn.py**.
5. Fine-tune the system parameters, network structure of these THREE networks.
6. Compare the training and testing performance of the THREE networks.
7. Save and submit all the codes & figures.

11.10 Summary

In this workshop, we learned about deep learning and CNNs.

We discussed what CNNs are and why we need them. We talked about the architecture of CNNs.

We learned about the various type of layers used within a CNN.

We discussed how to use TensorFlow. We used it to build a perceptron-based linear regressor.

We learned how to build an image classifier using a single-layer neural network.

We then built an image classifier using a CNN.

In the next workshop, we will learn about CNNs' other popular brother – Recurrent Neural Networks (RNNs).

Like CNNs, RNNs have taken flight and are extremely popular right now.

They have achieved impressive results against previous models; in some cases, in some cases, even surpassing human performance!!!

In []: